

```

        else if (abs(w[i]) > abs(w[i+1]))
            w[i]=abs(w[i])*sqrt(1.0+SQR(w[i+1]/w[i]));
        else w[i]=abs(w[i+1])*sqrt(1.0+SQR(w[i]/w[i+1]));
    }
    for (i=0;i<n;i++) r[0][i] += w[0]*v[i];
    for (i=0;i<k;i++)
        rotate(i,r[i][i],-r[i+1][i]);
    for (i=0;i<n;i++)
        if (r[i][i] == 0.0) sing=true;
}

void QRdcmp::rotate(const Int i, const Doub a, const Doub b)
Utility used by update. Given matrices r[0..n-1][0..n-1] and qt[0..n-1][0..n-1], carry
out a Jacobi rotation on rows i and i + 1 of each matrix. a and b are the parameters of the
rotation:  $\cos \theta = a/\sqrt{a^2 + b^2}$ ,  $\sin \theta = b/\sqrt{a^2 + b^2}$ .
{
    Int j;
    Doub c,fact,s,w,y;
    if (a == 0.0) {
        c=0.0;
        s=(b >= 0.0 ? 1.0 : -1.0);
    } else if (abs(a) > abs(b)) {
        fact=b/a;
        c=SIGN(1.0/sqrt(1.0+(fact*fact)),a);
        s=fact*c;
    } else {
        fact=a/b;
        s=SIGN(1.0/sqrt(1.0+(fact*fact)),b);
        c=fact*s;
    }
    for (j=i;j<n;j++) {
        y=r[i][j];
        w=r[i+1][j];
        r[i][j]=c*y-s*w;
        r[i+1][j]=s*y+c*w;
    }
    for (j=0;j<n;j++) {
        y=qt[i][j];
        w=qt[i+1][j];
        qt[i][j]=c*y-s*w;
        qt[i+1][j]=s*y+c*w;
    }
}

```

We will make use of *QR* decomposition, and its updating, in §9.7.

CITED REFERENCES AND FURTHER READING:

- Wilkinson, J.H., and Reinsch, C. 1971, *Linear Algebra*, vol. II of *Handbook for Automatic Computation* (New York: Springer), Chapter I/8.[1]
 Golub, G.H., and Van Loan, C.F. 1996, *Matrix Computations*, 3rd ed. (Baltimore: Johns Hopkins University Press), §5.2, §5.3, §12.5.[2]

2.11 Is Matrix Inversion an N^3 Process?

We close this chapter with a little entertainment, a bit of algorithmic prestidigitation that probes more deeply into the subject of matrix inversion. We start with a seemingly simple question:

How many individual multiplications does it take to perform the matrix multiplication of two 2×2 matrices,

$$\begin{pmatrix} a_{00} & a_{01} \\ a_{10} & a_{11} \end{pmatrix} \cdot \begin{pmatrix} b_{00} & b_{01} \\ b_{10} & b_{11} \end{pmatrix} = \begin{pmatrix} c_{00} & c_{01} \\ c_{10} & c_{11} \end{pmatrix} \quad (2.11.1)$$

Eight, right? Here they are written explicitly:

$$\begin{aligned} c_{00} &= a_{00} \times b_{00} + a_{01} \times b_{10} \\ c_{01} &= a_{00} \times b_{01} + a_{01} \times b_{11} \\ c_{10} &= a_{10} \times b_{00} + a_{11} \times b_{10} \\ c_{11} &= a_{10} \times b_{01} + a_{11} \times b_{11} \end{aligned} \quad (2.11.2)$$

Do you think that one can write formulas for the c 's that involve only *seven* multiplications? (Try it yourself, before reading on.)

Such a set of formulas was, in fact, discovered by Strassen [1]. The formulas are

$$\begin{aligned} Q_0 &\equiv (a_{00} + a_{11}) \times (b_{00} + b_{11}) \\ Q_1 &\equiv (a_{10} + a_{11}) \times b_{00} \\ Q_2 &\equiv a_{00} \times (b_{01} - b_{11}) \\ Q_3 &\equiv a_{11} \times (-b_{00} + b_{10}) \\ Q_4 &\equiv (a_{00} + a_{01}) \times b_{11} \\ Q_5 &\equiv (-a_{00} + a_{10}) \times (b_{00} + b_{01}) \\ Q_6 &\equiv (a_{01} - a_{11}) \times (b_{10} + b_{11}) \end{aligned} \quad (2.11.3)$$

in terms of which

$$\begin{aligned} c_{00} &= Q_0 + Q_3 - Q_4 + Q_6 \\ c_{10} &= Q_1 + Q_3 \\ c_{01} &= Q_2 + Q_4 \\ c_{11} &= Q_0 + Q_2 - Q_1 + Q_5 \end{aligned} \quad (2.11.4)$$

What's the use of this? There is one fewer multiplication than in equation (2.11.2), but *many more* additions and subtractions. It is not clear that anything has been gained. But notice that in (2.11.3) the a 's and b 's are never commuted. Therefore (2.11.3) and (2.11.4) are valid when the a 's and b 's are themselves matrices. The problem of multiplying two very large matrices (of order $N = 2^m$ for some integer m) can now be broken down recursively by partitioning the matrices into quarters, sixteenths, etc. And note the key point: The savings is not just a factor "7/8"; it is that factor at *each* hierarchical level of the recursion. In total it reduces the process of matrix multiplication to order $N^{\log_2 7}$ instead of N^3 .

What about all the extra additions in (2.11.3) – (2.11.4)? Don't they outweigh the advantage of the fewer multiplications? For large N , it turns out that there are six times as many additions as multiplications implied by (2.11.3) – (2.11.4). But, if N is very large, this constant factor is no match for the change in the *exponent* from N^3 to $N^{\log_2 7}$.

With this “fast” matrix multiplication, Strassen also obtained a surprising result for matrix inversion [1]. Suppose that the matrices

$$\begin{pmatrix} a_{00} & a_{01} \\ a_{10} & a_{11} \end{pmatrix} \quad \text{and} \quad \begin{pmatrix} c_{00} & c_{01} \\ c_{10} & c_{11} \end{pmatrix} \quad (2.11.5)$$

are inverses of each other. Then the c ’s can be obtained from the a ’s by the following operations (compare equations 2.7.11 and 2.7.25):

$$\begin{aligned} R_0 &= \text{Inverse}(a_{00}) \\ R_1 &= a_{10} \times R_0 \\ R_2 &= R_0 \times a_{01} \\ R_3 &= a_{10} \times R_2 \\ R_4 &= R_3 - a_{11} \\ R_5 &= \text{Inverse}(R_4) \\ c_{01} &= R_2 \times R_5 \\ c_{10} &= R_5 \times R_1 \\ R_6 &= R_2 \times c_{10} \\ c_{00} &= R_0 - R_6 \\ c_{11} &= -R_5 \end{aligned} \quad (2.11.6)$$

In (2.11.6) the “inverse” operator occurs just twice. It is to be interpreted as the reciprocal if the a ’s and c ’s are scalars, but as matrix inversion if the a ’s and c ’s are themselves submatrices. Imagine doing the inversion of a very large matrix, of order $N = 2^m$, recursively by partitions in half. At each step, halving the order *doubles* the number of inverse operations. But this means that there are only N divisions in all! So divisions don’t dominate in the recursive use of (2.11.6). Equation (2.11.6) is dominated, in fact, by its 6 multiplications. Since these can be done by an $N^{\log_2 7}$ algorithm, so can the matrix inversion!

This is fun, but let’s look at practicalities: If you estimate how large N has to be before the difference between exponent 3 and exponent $\log_2 7 = 2.807$ is substantial enough to outweigh the bookkeeping overhead, arising from the complicated nature of the recursive Strassen algorithm, you will find that LU decomposition is in no immediate danger of becoming obsolete. However, the fast matrix multiplication routine itself is beginning to appear in libraries like BLAS, where it is typically used for $N \gtrsim 100$.

Strassen’s original result for matrix multiplication has been steadily improved. The fastest currently known algorithm [2] has an asymptotic order of $N^{2.376}$, but it is not likely to be practical to implement it.

If you like this kind of fun, then try these: (1) Can you multiply the complex numbers $(a + ib)$ and $(c + id)$ in only *three* real multiplications? [Answer: See §5.5.] (2) Can you evaluate a general fourth-degree polynomial in x for many different values of x with only *three* multiplications per evaluation? [Answer: See §5.1.]

CITED REFERENCES AND FURTHER READING:

Strassen, V. 1969, “Gaussian Elimination Is Not Optimal,” *Numerische Mathematik*, vol. 13, pp. 354–356.[1]